Robotic Manipulation

Perception, Planning, and Control

Russ Tedrake

© Russ Tedrake, 2020-2026
Last modified 2026-9-13.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Fall 2026 semester.

Table of contents Next Chapter

Chapter 1
Introduction

It's worth taking time to appreciate just how amazing humans are when performing tasks with our hands. Tasks that often feel mundane to us -- loading the dishwasher, chopping vegetables, folding laundry -- remain as incredibly challenging tasks for robots and are at the very forefront of robotics research.



Consider the problem of picking up a single plate from a stack of plates in the sink and placing it into the dishwasher. Clearly you first have to perceive that there is a plate in the sink and that it is accessible. Getting your hand to the plate likely requires navigating your hand around the geometry of the sink and other dishes. The act of actually picking it up might require a fairly subtle maneuver where you have to tip up the plate, sliding it along your fingers and then along the sink/dishes in order to get a reasonable grasp on it. Presumably as you lift it out of the sink, you'd like to mostly avoid collisions between the plate and the sink, which suggests a reasonable understanding of the size/extent of the plate (though I actually think robots today are too afraid of collisions). Even placing the plate into the dishwasher is pretty subtle. You might think that you would align the plate with the slats and then slide it in, but I think humans are more clever than that. A seemingly better strategy is to loosen your grip on the plate, come in at an angle and intentionally contact one side of the slat, letting the plate effectively rotate itself into position as you set it down. But the

justification for this strategy is subtle -- it is a way to achieve the kinematically accurate task without requiring much kinematic accuracy on the position/orientation of the plate.



Figure 1.2 - A robot picking up a plate from a potentially cluttered sink (left: in simulation, right: in reality). This example is from the manipulation team at the Toyota Research Institute.

Perhaps one of the reasons that these problems remain so hard is that they require strong capabilities in numerous technical areas that have traditionally been somewhat disparate; it's challenging to be an expert in all of them. More so than robotic mapping and navigation, or legged locomotion, or other great areas in robotics, the most interesting problems in manipulation require significant interactions between perception, planning, and control. This includes both geometric perception to understand the local geometry of the objects and environment and semantic perception to understand what opportunities for manipulation are available in the scene. Planning typically includes reasoning about the kinematic and dynamic constraints of the task (how do I command my rigid seven degree-of-freedom arm to reach into the drawer?). But it also includes higher-level task planning (to get milk into my cup, I need to open the fridge, then grab the bottle, then unscrew the lid, then pour... and perhaps even put it all back when I'm done). The low-level begs for representations using real numbers, but the higher levels feel more logical and discrete. At perhaps the lowest level, our hands are making and breaking contact with the world either directly or through tools, exerting forces, rapidly and easily transitioning between sticking and sliding frictional regimes -- these alone are incredibly rich and difficult problems from the perspective of dynamics and control.

There is a lot for us to discuss!

1.1 Manipulation is more than pick-and-place

There are a large number of applications for manipulation. Picking up an object from one bin and placing it into another bin -- one version of the famous "pick and place" problem -- is a great application for robotic manipulation. Robots have done this for decades in factories with carefully curated parts. In the last few years, we've seen a new generation of pick-and-place systems that use deep learning for perception, and can work well with much more diverse sets of objects, especially if the

location/orientation of the placement need not be very accurate. This can be done with conventional robot hands or more special-purpose end-effectors that, for instance, use suction. It can often be done without having a very accurate understanding of the shape, pose, mass, nor friction of the object(s) to be picked.

The goal for these notes, however, is to examine the much broader view of manipulation than what is captured by the pick and place problem. Even our thought experiment of loading the dishwasher -- arguably a more advanced type of pick and place -- requires much more from the perception, planning, and control systems. But the diversity of tasks that humans (and hopefully soon robots) can do with their hands is truly remarkable. To see one small catalog of examples that I like, take a look at the Southampton Hand Assessment Procedure (SHAP), which was designed as a way to empirically evaluate prosthetic hands. Matt Mason also gave a broad and thoughtful definition of manipulation in the opening of his 2018 review paperMason18.

It's also important to recognize that manipulation research today looks very different than manipulation research looked like in the 1980s and 1990s. During that time there was a strong and beautiful focus on "manipulation as grasping," with seminal work on, for instance, the kinematics/dynamics of multi-fingered hands assuming a stable grasp on an object. Sometimes still, I hear people use the term "grasping" as almost synonymous with manipulation. But please realize that the goals of manipulation research today, and of these notes, are much broader than that. Is grasping a sufficient description of what your hand is doing when you are buttoning your shirt? Making a salad? Spreading peanut butter on toast?

1.2 Open-world manipulation

Perhaps because humans are so good at manipulation, our expectations in terms of performance and robustness for these tasks are extremely high. It's not enough to be able to load one set of plates in a laboratory environment into a dishwasher reliably. We'd like to be able to manipulate basically any plate that someone might put in the sink. And we'd like the system to be able to work in any kitchen, despite various geometric configurations, lighting conditions, etc. The challenge of achieving and verifying robustness of a complex manipulation stack with physics, perception, planning, and control in the loop is already daunting. But how do we provide test coverage for every kitchen in the world?

The idea that the world has infinite variability (there will never be a point at which you have seen every possible kitchen) is often referred to as the "open-world" or "open-domain" problem -- a term popularized first in the context of video games. It can be tough to strike a balance between rigorous thinking about aspects of the manipulation problem, and trying to embrace the diversity and complexity of the entire world. But it's important to walk that line.

There is a chance that the diversity of manipulation in the open world might actually make the problem easier. We are just at the dawn of the era of big data in robotics; the impact this will have cannot be overstated. But I think it is deeper than that. As an example, some of our optimization formulations for planning and control might get stuck in local minima now because narrow problem formulations can have many quirky solutions; but once we ask a controller to work in a huge diversity of scenarios then the quirky solutions can be discarded and the optimization landscape may become much simpler. But to the extent that is true, then we should aim to understand it rigorously!

1.3 Simulation

There is another reason that we are now entering a golden age for manipulation research. Our software tools have (very recently) gotten much better!

I remember a just few years ago (~2015) talking to my PhD students, who were all quite adept at using simulation for developing control systems for walking robots, about using simulation for manipulation. "You can't work on manipulation in simulation" was their refrain, and for good reason. The complexity of the contact mechanics in manipulation has traditionally been much harder to simulate than a walking robot that only interacts with the ground and through a minimal set of contact points. Looming even larger, though, was the centrality of perception for manipulation; it was generally accepted that one could not simulate a camera well enough to be meaningful.

How quickly things can change! The last few years has seen a rapid adoption of video-game quality rendering by the robotics and computer vision communities. The growing consensus now is that game-engine renderers *can* model cameras well enough not only to *test* a perception system in simulation, but even to *train* perception systems in simulation and expect them to work in the real world! This is fairly amazing, as we were all very concerned before that training a deep learning perception system in simulation would allow it to exploit any quirks of the simulated images that could make the problem easier.

We have also seen dramatic improvements in the quality and performance of contact simulation. Making robust and performant simulations of multi-body contact involves dealing with complex geometry queries and stiff (measure-) differential equations. There is still room for fundamental improvements in the mathematical formulations of and numerical solutions for the governing equations, but today's solvers are good enough to be extremely useful.

1.4 These notes are interactive

By leveraging the power of simulation, and the new availability of free online interactive compute, I am trying to make these notes more than what you would find in a traditional text. Each chapter will have working code examples, which you can run

in Google Colab, or install and run on your local machine (see the appendix for more details). It uses an open-source library called which has been a labor of love for me since around 2013 when I started taking our research code and making it more broadly available. Because all of the code is open source, it is entirely up to you how deep into the rabbit hole you choose to go.

Mortimer Adler famously said "Reading a [great] book should be a conversation between you and the author." In addition to the interactive graphics/code, I've added the ability to highlight/comment/ask questions directly on these notes (go ahead and try it; but please read my note on annotation etiquette). Adler's suggestion was that *great* writing can turn static text into a dialogue, transcending distance and time; perhaps I'm cheating, but technology can help me communicate with you even if my writing isn't as strong as Adler would have liked! Adler also recommends writing on your booksAdler41; you can print the pages, use the (infrequently updated) pdf, or make private annotations right on the website using the same annotation tool.

I have organized the software examples into notebooks by chapter. Click the "Open in Colab" button beside an example to open its notebook, then run the cells in order. The first code cell installs the required packages in Colab; it does nothing when you run the notebook locally. As you read the text, the examples correspond to sections in the notebooks.

Example 1.1 (Teleoperation in 2D.)

Before we get into any autonomous manipulation, let's start by just getting a feel for what it will be like to work on manipulation in an online Jupyter notebook. This example will open up a new window with our 3D visualizer, and in the "Controls" menu in the visualizer, you will find sliders that you can use to drive the end-effector of the robot around. Give it a spin!

Open in Colab

Example 1.2 (Teleoperation in 3D.)

I offered a 2D visualization / control first because everything is simpler in 2D. But the simulation is actually running fully in 3D. Run the second example to see it.

Open in Colab

1.5 Model-based design and analysis

Thanks to progress in simulation, it is now possible to pursue a meaningful study of manipulation without needing a physical robot. But the software advances in simulation (of our robot dynamics, sensors, actuators and its environment) are not enough to support all of the topics in these notes. Manipulation research today

leverages a myriad of advanced algorithms in perception, planning, and control. In addition to providing those individual algorithms, a major goal of these notes is to attempt to bridle the complexity of using them together in a systematic way.

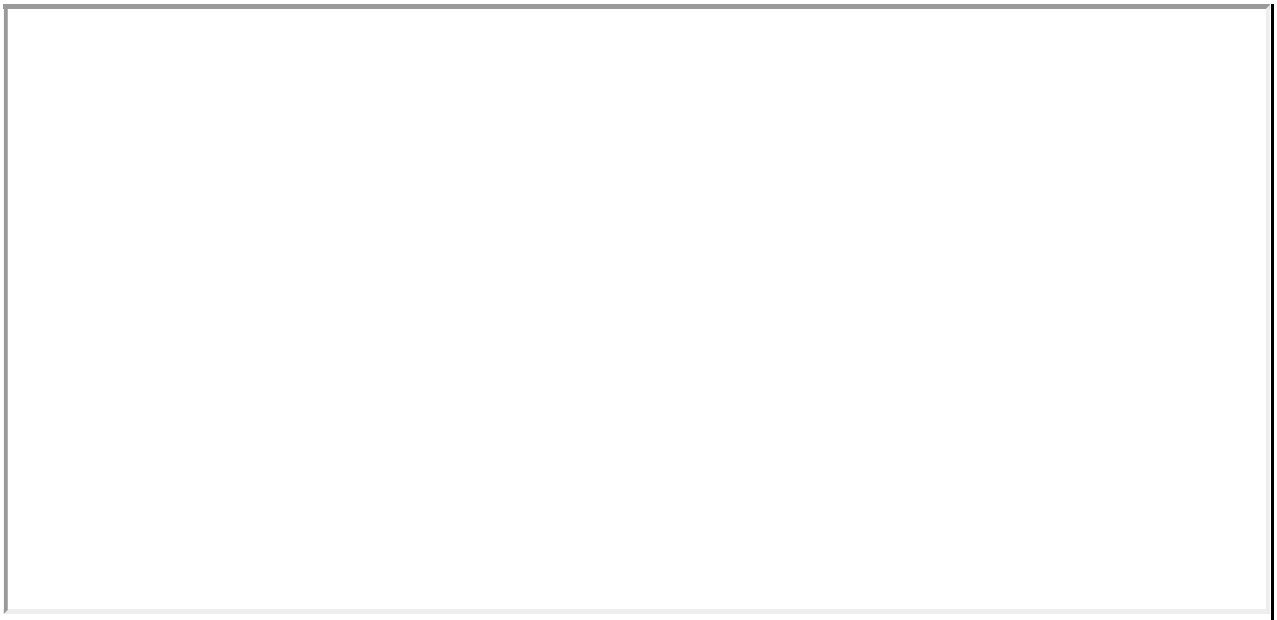
Many of you will be familiar with ROS (the Robot Operating System). I believe that ROS was one of the best things to happen to robotics in the last decades. It meant that experts from different subdisciplines could easily share their expertise in the form of modular components. Components (as ROS packages) simply agree on the messages that they will send and receive on the network; packages can inter-operate even if they are written in different programming languages or even on different operating systems.

Although ROS makes it relatively easy to get started with manipulation, it doesn't serve my pedagogical goal of thinking clearly about manipulation. The modular approach to authoring the components is extremely good, and we will adopt it here. But in we ask for a little bit more from each of the components -- essentially that they declare their states, parameters, and timing semantics in a consistent way -- so that we have a much better chance of understanding the complex relationships between systems. This has great practical value as well; the ability to debug a full manipulation stack with repeatable deterministic simulations (even if they include randomness) is surprisingly rare in the field but hugely valuable.

The key building block in our work will be **Systems**, and systems can be combined in complex combinations into **Diagrams**. System diagrams have long been the modeling paradigm used in controls, and the software aspect of it will be very familiar to you if you've used tools like Simulink, LabView, or Modelica. These software tools refer to the block-diagram design paradigm as "model-based design".

Example 1.3 (System Diagrams)

Even the examples above, which relied on you to perform teleoperation instead of having an autonomy stack, were still the result of combining numerous parts. For any system (a system diagram is still a system) that you have in , you can visualize that diagram directly in the notebook.



This graphic is interactive. Make sure you zoom in and click around to get a sense for the amount of complexity that we can abstract away in this framework. For instance, try expanding the `iiwa_controller` block.

Whenever you are ready to learn more about the Systems Framework in , I recommend starting with the "Modeling Dynamical Systems" tutorial linked from the main Drake website.

Let me be transparent: not everybody likes this systems framework for manipulation. Some people are just trying to write code as fast as possible, and don't see the benefits of slowing down to declare their state variables, etc. It will likely feel like a burden to you the first time you go to write a new system. But it's precisely *because* we're trying to build complex systems quickly that I advocate this more rigorous approach. I believe that getting to the next level of maturity in our open-world manipulation systems requires more maturity from our building blocks.

1.6 Organization of these notes

The remaining chapters of these notes are organized around the component-level building blocks of manipulation. Many of these components each individually build on a wealth of literature (e.g. from computer vision, or dynamics and control). Rather than be overwhelmed, I've chosen to focus on delivering a consistent coherent presentation of the most relevant ideas from each field *as they relate to manipulation*, and pointers to more literature. Even finding a single notation across all of the fields can be a challenge!

The next few chapters will give you a minimal background on the relevant robot hardware that we are simulating, on (some of) the details about simulating them, and on the geometry and kinematics basics that we will use heavily through the notes.

For the remainder of the notes, rather than organize the chapters into "Perception", "Planning", and "Control", I've decided to spiral through these topics. In the first part, we'll do just enough perception, planning, and control to build up a basic manipulation system that can perform pick-and-place manipulation of known isolated objects. Then I'll try to motivate each new chapter by describing what our previous system cannot do, and what we hope it will do by the end of the chapter.

I welcome any feedback. And don't forget to interact!

1.7 Exercises

Exercise 1.1 Familiarize yourself with Drake

is a powerful and mature software library that can support many advanced robotics applications. The motivation behind it is described in this blog post. It is extensively documented, but you have to know where to look for it.

1. Check out the growing list of tutorials (linked from the main Drake page). In the `dynamical_systems` tutorial, to what value is the initial condition, $x(0)$, set when we simulate the `SimpleContinuousTimeSystem`?
2. The class-/function-level documentation is the most extensive documentation in Drake. When I'm working on Drake (in either C++ or Python), I most often have the C++ doxygen open. The Python documentation is (mostly) auto-generated from this and isn't curated as carefully; I tend to look there only in the rare cases that the Python interface differs from C++.
   In C++ doxygen, search for "Spatial Vectors". What ascii characters do we use to denote an angular acceleration in code?
3. Drake is open-source. There are no black-box algorithms here. If you ever want to see how a particular algorithm is implemented, or find examples of how to use a function, you can always look at the source code. These days you can use VS Code to explore the code right in your browser. What value of `convergence_tol` do I use in the unit test for "fitted value iteration"?

As you use Drake, if you have any questions, please consider asking them on stack**overflow** using the "drake" tag. The broader Drake developers community will often be able to answer faster (and/or better) than the course staff! And asking there helps to build a searchable knowledge base that will make Drake more useful and accessible for everyone.

Exercise 1.2 Drake Systems Fundamentals

This exercise will introduce you to Drake's block-diagram systems framework, and we will work with some core concepts in Drake including `LeafSystem`, `Diagram`, `Context`, and `Simulator`. For this exercise you will implement a custom dynamical system from scratch using Drake's systems framework. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Implement a custom `LeafSystem` for an inverted pendulum
b. Build and wire systems into a `Diagram` for simulation
c. Run simulations using Drake's built-in tools

Exercise 1.3 Physics Simulation and Robot Loading

Building on the systems framework from the previous exercise, you'll now learn how to set up a scene with robots and custom assets. Specifically, this exercise will show you how to load the Kuka iiwa robot from Drake's model database and create your own custom objects by writing `SDF` files from scratch. By the end, you'll have a simulation where your custom initials fall onto a table you created, with a 7-DOF Kuka iiwa robot in the scene! You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Load the Kuka iiwa14 robot into a `Diagram` using `Parser` and `MultibodyPlant`
b. Implement a simple proportional controller as a `LeafSystem` that computes joint torques to position the robot at a desired joint configuration
c. Author a custom `SDF` file to define a table, then generate `SDF` assets for your initials using a provided letter generation API
d. Assemble a simulation with the robot, your custom table, and your initials

Exercise 1.4 HardwareStation and Advanced Scenarios

In the previous exercise, you manually set up a simulation by creating a `DiagramBuilder`, adding a `MultibodyPlant` and `SceneGraph`, loading robots, and wiring everything together. In this exercise, you will learn to use `HardwareStation` and model directives specified in `YAML` configuration files to achieve the same result with much less code. You will work exclusively in this notebook. You will complete the following steps:

a. Set up a scene with two iiwa14 robots using `HardwareStation` and scenario directives
b. Load your custom table and letter assets into `HardwareStation`
c. Add pre-defined objects from Drake's model repository into `HardwareStation`

References

1. Matthew T Mason, "Toward robotic manipulation", *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, pp. 1--28, 2018.

2. Mortimer J Adler, "How to Mark a Book", *The Saturday Review of Literature*, July 6, 1941. [link]

Table of contents Next Chapter

Accessibility

© Russ Tedrake, 2026